

P4: Client/Server BattleShipGame

Abgabetermin: 8. Mai 2016

Punktzahl: XX

In diesem Projekt wird die Kommunikation zwischen einem Server und mehreren Clients am Beispiel des Spiels **Schiffeversenken** umgesetzt. Der Fokus liegt dabei ausschließlich auf der **Netzwerkcommunication** mittels definierter **Protokoll-Nachrichten** – eine konkrete Spiellogik oder grafische Benutzeroberfläche wird nicht implementiert.

Zunächst werden dazu die notwendigen **Nachrichtenklassen (Messages)** implementiert, die den Austausch von Informationen wie Spielbeitritten, Warteschlangen-Updates oder Fehlermeldungen ermöglichen (Aufgabe 1). Anschließend folgt die Umsetzung eines **Socket-Servers**, der über eigene **Threads** mehrere Clients gleichzeitig bedienen kann und über ein festgelegtes **Server-Protokoll** eingehende Nachrichten korrekt verarbeitet (Aufgabe 2). Schließlich wird auf Client-Seite ein passendes **Client-Protokoll** erstellt, das die empfangenen Nachrichten des Servers annimmt, interpretiert und an spezialisierte Handler zur weiteren Verarbeitung weiterleitet (Aufgabe 3).

Ziel ist es, ein funktionierendes **Kommunikationssystem** zu schaffen, das die Grundlage für ein späteres Netzwerkspiel bildet, wobei die Entwicklung auf die reine Netzwerkstruktur begrenzt bleibt und keine Spielmechanik oder grafische Darstellung umfasst.

Sie finden in ILIAS die Archivdatei **P4.zip** vor, die Sie in *eclipse* als neues Projekt importieren können.

Vorgaben

Die Vorgaben unter **P4.zip** definieren bereits eine Grundstruktur, die Sie für Ihre Implementierung verwenden sollen. Darin sind mehrere Packages vorgegeben, die nachfolgend kurz erläutert werden.

Package server.*

Spielprinzip und Spielablauf

Im Spiel **Schiffeversenken** platzieren zwei Spieler zunächst verdeckt Schiffe unterschiedlicher Größen auf einem quadratischen Spielfeld, welches aus einem Raster besteht. Ziel des Spiels ist es, alle Schiffe des Gegners zu versenken. Dazu wählen die Spieler abwechselnd Felder aus, indem sie deren Koordinaten angeben (z. B. "B5"). Ein Treffer wird rot markiert, während ein Fehlschuss grau erscheint. Trifft ein Spieler ein Schiff, erhält er einen weiteren Zug. Das Spiel endet, sobald entweder alle gegnerischen Schiffe versenkt sind, ein Spieler aufgibt oder ein Spieler die Verbindung zum Server verliert.

Zusätzlich verfügt das Spiel über ein Energiesystem: Spieler erhalten für erfolgreiche Züge und Treffer Energiepunkte, welche sie einsetzen können, um spezielle Items zu aktivieren, wie beispielsweise eine *Seabomb* (trifft ein 2x2-Feld), ein *AirStrike*, der eine komplette Zeile oder Spalte trifft, oder ein *Radar*, das die Anzahl der Schiffe in einem 3x3 Bereich anzeigt.

Server-Komponente

Server
...
+ startServer() : void + registerGame(game : BattleShipGame) : void + unregisterGame(id : UUID) : void + getGame(player : ServerPlayer) : BattleShipGame + getGameFromJoinCode(joinCode : int) : BattleShipGame + addToQueue(player : ServerPlayer) : void + removeFromQueue(id : UUID) : void

Beschreibung der Klasse Server

Die Klasse **Server** verwaltet die gesamte Kommunikation zwischen Clients und dem Spielserver. Sie nimmt neue Verbindungen entgegen, erstellt eigene Threads für jeden Spieler und organisiert aktive Spiele sowie eine Warteschlange für Spieler. Außerdem bietet sie Methoden zum Registrieren und Entfernen von Spielen sowie zur Verwaltung

der Spielsuche (Queue). Damit bildet sie das zentrale Steuerungselement für den Mehrspielermodus.

Game (Interface)

<i>Game</i>
«interface»
+ addPlayer(player : ServerPlayer) : void + removePlayer(player : ServerPlayer) : void + onPlayerReadyStateChange(player : ServerPlayer, ready : boolean) : void + onPlayerPlaceShips(player : ServerPlayer, ships : ArrayList<Ship>) : void + onPlayerAttemptMove(player : ServerPlayer, move : Move) : void + sendGameStartingEvent() : void + sendInGameStartEvent() : void + sendTurnChangeEvent() : void + sendGameOverEvent(reason : GameOverReason) : void + initAvailableShips() : void + getGameState() : GameState + setGameState(gameState : GameState) : void

Beschreibung des Interface Game

Das Interface **Game** legt alle Methoden fest, die zur Verwaltung eines Spiels notwendig sind. Es wird ausschließlich von der Klasse **BattleShipGame** implementiert. Dazu gehören unter anderem das Hinzufügen und Entfernen von Spielern, das Verarbeiten von Spielzügen und das Versenden von Spielereignissen. Damit bildet das Interface die Grundlage für die gesamte Spielsteuerung von **BattleShipGame**.

GameOptions

GameOptions
...
+ GameOptions() : void + setBuildTime(BUILD_TIME : int) : void + setMoveTime(MOVE_TIME : int) : void + setMoveHitTimeBonus(MOVE_HIT_TME_BONUS : int) : void + setEnergyGameStart(ENERGY_GAME_START : int) : void + setEnergyTurnBonus(ENERGY_TURN_BONUS : int) : void + setBoardSize(BOARD_SIZE : int) : void + setEnergyShipHit(ENERGY_SHIP_HIT : int) : void + getBoardSize() : int, getBuildTime() : int, ...

Beschreibung der Klasse GameOptions

Die Klasse **GameOptions** legt die Rahmenbedingungen eines Spiels fest, wie beispielsweise die Spielfeldgröße, Bauzeit, Zugzeit und Energiesystem. Sie wird in jedem **GameState** gespeichert und definiert damit die Spielregeln für ein bestimmtes Spiel. Beim Erstellen eines benutzerdefinierten Spiels sendet der Client ein **GameOptions**-Objekt über das **CreateGameEvent**, um dem Server die gewünschten Spielparameter mitzuteilen.

GameContainer

GameContainer
...
+ GameContainer(game : BattleShipGame, thread : Thread) : void + getGame() : BattleShipGame + getThread() : Thread

Beschreibung der Klasse GameContainer

Die Klasse **GameContainer** verwaltet die Spielinstanz eines **BattleShipGame** sowie den zugehörigen Thread, auf dem das Spiel läuft. Sie wird vom Server genutzt, um aktive Spiele und deren Threads gemeinsam zu speichern und zu verwalten. Über entsprechende Getter-Methoden kann sowohl auf das Spiel als auch auf den Thread zugegriffen werden, um z. B. ein Spiel zu beenden oder zu verwalten.

ServerPlayer

ServerPlayer
...
+ ServerPlayer(socket : Socket, server : Server) + run() : void + sendMessage(message : Message) : void + getUsername() : String + getIp() : String + getId() : UUID + isInGame() : boolean + setInGame(inGame : boolean) : void – broadcastQueueStateUpdate() : void – checkQueueForPossibleGame() : void – createGame(gameOptions : GameOptions) : void – createGame(playerA : ServerPlayer, playerB : ServerPlayer) : void

Beschreibung der Klasse ServerPlayer

Die Klasse **ServerPlayer** stellt die Server-seitige Spieler-Instanz dar und verwaltet die Kommunikation mit einem verbundenen Client. Für jeden Spieler wird ein eigener Thread erstellt, der alle eingehenden Nachrichten vom Client empfängt, verarbeitet und entsprechende Antworten versendet. Über diese Klasse werden zudem Spielaktionen wie das Betreten einer Warteschlange, das Erstellen oder Beitreten von Spielen sowie Spielzüge abgewickelt und an die jeweiligen Spielinstanzen übergeben.

Message

<i>Message</i>
type : MessageType
Message(type : MessageType) + getType() : MessageType

Beschreibung der abstrakten Klasse Message

Zentrales Element für die Kommunikation zwischen Server und Client ist ein eigens definiertes **Nachrichtenprotokoll** im Package `protocol.messages`. Dieses Protokoll basiert auf einer abstrakten Basisklasse **Message**, von der alle spezifischen Nachrichtenarten erben.

Richtung	MessageType
Server → Client	REGISTER QUEUE_UPDATE JOIN_GAME BUILDING_PHASE_STARTS BUILD_READY_STATE_CHANGE GAME_IN_GAME_START TURN_CHANGE MOVE_MADE GAME_OVER ERROR
Client → Server	JOIN_QUEUE LEAVE_QUEUE CREATE_GAME JOIN_GAME_WITH_CODE PLAYER_UPDATE_SHIP_PLACEMENT PLAYER_READY PLAYER_MOVE LEAVE_GAME
Client → Server; Server → Client	PLAYER_HOVER

Die einzelnen **MessageType**-Einträge definieren dabei den genauen Typ einer Nachricht und bestimmen, wie diese zwischen Server und Client verarbeitet werden muss.

Jede konkrete Nachricht erbt von der abstrakten Basisklasse **Message** und implementiert zusätzlich spezifische Informationen, die für die jeweilige Aktion benötigt werden. Anhand des **type**-Attributs in der Basisklasse kann zur Laufzeit effizient entschieden werden, welche Nachricht empfangen wurde und wie darauf reagiert werden soll.

Package protocol.*

Geteilte Klassen

GameState

GameState
...
+ getId() : UUID + getSessionCode() : int + getAvailableShips() : ArrayList<Ship> + getStatus() : GameStatus + getPlayerA() : ClientPlayer + getPlayerB() : ClientPlayer + getBoardSize() : int + getPlayersTurnStart() : Date + getPlayersTurnEnd() : Date + getBuildGameBoardStarted() : Date + getBuildGameBoardFinished() : Date + getGameOptions() : GameOptions

Beschreibung der Klasse GameState

Die Klasse **GameState** bildet den aktuellen Spielstand eines Spiels ab und enthält alle wichtigen Informationen zum Spielverlauf. Dazu gehören unter anderem die beiden Spieler, die verfügbaren Schiffe, der aktuelle **GameState** das Spielfeld sowie der aktuelle Status des Spiels. Der **GameState** wird auf dem Server verwaltet und bei jeder relevanten Änderung an die verbundenen Clients übertragen, sodass beide Spieler stets den aktuellen Stand des Spiels sehen können. Neben Informationen zu Spielbeginn, Spielende und Rundenwechsel enthält die Klasse auch Methoden zur Bestimmung von getroffenen und versunkenen Schiffen. Über Getter können alle Spiel-relevanten Zustände wie Spieler, Spielstatus und Spieloptionen abgefragt werden. Der **GameState** dient damit als zentrales Datenelement zur Synchronisation des Spiels zwischen Server und Client.

Beschreibung des Enums GameState

Das Enum `GameState` definiert die verschiedenen Phasen, die ein Spiel während seines Ablaufs durchläuft. Es dient dazu, den aktuellen Zustand des Spiels eindeutig festzuhalten und den Spielfluss zu steuern. Der Status `LOBBY_WAITING` zeigt an, dass sich das Spiel noch in der Lobby befindet und auf Spieler wartet. In der Phase `BUILD_GAME_BOARD` platzieren die Spieler ihre Schiffe auf dem Spielfeld. Sobald das Spiel gestartet ist, wechselt der Status zu `IN_GAME`, und das Spiel läuft aktiv. Nach Spielende wird der Status auf `GAME_OVER` gesetzt, was anzeigt, dass das Spiel abgeschlossen ist und ein Gewinner feststeht.

GameState	Beschreibung
LOBBY_WAITING	Der Wartestatus vor Spielbeginn. Spieler treten der Lobby bei und warten, bis alle bereit sind.
BUILD_GAME_BOARD	Aufbauphase: Die Spieler platzieren ihre Schiffe auf dem Spielfeld.
IN_GAME	Das Spiel läuft. Spieler führen ihre Züge abwechselnd aus und versuchen, die gegnerischen Schiffe zu treffen.
GAME_OVER	Das Spiel ist beendet. Ein Spieler hat gewonnen oder es gibt keinen möglichen Zug mehr.

ClientPlayer

ClientPlayer	
...	
+ getId() : UUID	+ isInGame() : boolean
+ getName() : String	+ isWinner() : boolean
+ getEnergy() : int	+ isPlayer(id : UUID) : boolean
+ getMoves() : ArrayList<Move>	
+ getUncoveredShips() : ArrayList<Ship>	
+ isTurn() : boolean	
+ isReady() : boolean	

Beschreibung der Klasse ClientPlayer

Die Klasse `ClientPlayer` stellt die individuellen Spielerdaten innerhalb eines Spiels dar und wird vom `GameState` für beide Spieler genutzt. Sie speichert wichtige Informationen wie die eindeutige Spieler-ID, den Spielernamen, die bereits ausgeführten Züge (`moves`),

aufgedeckte Schiffe (**uncoveredShips**), den aktuellen Energiewert sowie den Spielstatus (z.B. ob der Spieler noch im Spiel ist, ob er am Zug ist oder gewonnen hat). Damit erlaubt die Klasse eine klare Trennung zwischen allgemeinen Spieldaten und den individuellen Zuständen der einzelnen Spieler. **ClientPlayer** wird auch genutzt, um den aktuellen Stand eines Spielers bei Synchronisierungen zwischen Server und Client zu übertragen. Durch die Methoden können unter anderem der Energie- und Schiffsstatus abgefragt werden.

Ship

Ship
...
+ getOccupiedCells() : ArrayList<Point> + getOccupiedCellsAt(x : int, y : int) : ArrayList<Point> + getOrientation() : Orientation + getX() : int + getY() : int + getId() : int + getLength() : int + getWidth() : int

Beschreibung der Klasse Ship

Die Klasse **Ship** repräsentiert ein einzelnes Schiff innerhalb des Spiels und speichert alle relevanten Eigenschaften wie Position, Orientierung, Länge und Breite des Schiffs. Sie verwaltet außerdem, ob ein Schiff bereits platziert oder versenkt wurde und wie viele Treffer es bereits erhalten hat. Über Methoden wie **getOccupiedCells()** kann ermittelt werden, welche Spielfelder ein Schiff belegt. Damit ermöglicht die Klasse eine präzise Berechnung von Trefferflächen und unterstützt die Spiellogik bei der Platzierung und Treffererkennung. Zudem erlaubt die Enumeration **Orientation**, die Ausrichtung eines Schiffs dynamisch zu verändern. Insgesamt ist **Ship** eine zentrale Komponente zur Verwaltung der Schiffe beider Spieler im Spielablauf.

Client-Komponente

Die **Client-Komponente** stellt die Schnittstelle dar, über die Spieler mit dem Spiel interagieren. Dabei übernimmt sie zwei wesentliche Funktionen: Zum einen die Kommunikation mit dem Server, und zum anderen die Aufbereitung der erhaltenen Nachrichten

zur weiteren Verarbeitung. Der Client setzt sich aus mehreren Klassen zusammen, die jeweils spezifische Aufgaben erfüllen.

ClientHandler

ClientHandler
...
+ ClientHandler(serverAddress : String, port : int) + sendMessage(message : Message) : void + listenForMessages() : void + getGameHandler() : GameHandler + getLobbyHandler() : LobbyHandler + showError(message : String) : void + endCurrentGame() : void

Eine zentrale Rolle spielt die Klasse **ClientHandler**, die die Verbindung zum Server herstellt und die Streams zur Kommunikation initialisiert. Sie ist dafür zuständig, eingehende Nachrichten vom Server zu empfangen und diese an die jeweiligen spezialisierten Komponenten, wie den Lobby- oder GameHandler, weiterzuleiten.

LobbyClient

<i>LobbyClient</i>
«interface»
+ onQueueUpdate(queueUpdateMessage : QueueUpdateMessage) : void + onLobbyError(errorMessage : ErrorMessage) : void + sendJoinQueueEvent() : void + sendLeaveQueueEvent() : void + sendCreateGameEvent(gameOptions : GameOptions) : void + sendJoinGameWithCodeEvent(code : int) : void

Die Klasse **LobbyHandler** (Interface client.LobbyClient.java) übernimmt alle Aufgaben rund um die Interaktion in der Lobby, darunter das Beitreten und Verlassen der Warteschlange sowie das Erstellen oder Betreten von Spielen. Sie kommuniziert direkt mit dem **ClientHandler** und sorgt dafür, dass Nachrichten aus der Lobby korrekt verarbeitet werden.

GameClient

<i>GameClient</i>
«interface»
+ onPlayerHoverEvent(message : PlayerHoverMessage) : void + onBuildPhaseStarts(message : GameBuildingStartMessage) : void + onGameInGameStarts(message : GameInGameStartMessage) : void + onGameOver(message : GameOverMessage) : void + onTurnChange(message : PlayerTurnChangeMessage) : void + onMoveMade(message : MoveMadeMessage) : void + onBuildReadyStateChange(message : PlayerReadyStateChangeMessage) : void + onGameError(message : ErrorMessage) : void + showBuildPhase() : void + sendPlayerHoverEvent(x : int, y : int, affectedFields : ArrayList<Cell>) : void + sendUpdateBuildBoardMessage(ships : ArrayList<Ship>) : void + sendPlayerReadyMessage(ready : boolean) : void + sendGameMoveEvent(move : Move) : void + sendLeaveGame() : void

Schließlich verwaltet die Klasse **GameHandler** (Interface client.GameClient.java) alle spielbezogenen Interaktionen. Dazu gehören insbesondere das Platzieren der Schiffe in der Bauphase, der Wechsel von Spielzügen, die Nutzung von Items und das Erkennen von Spielereignissen wie dem Zugwechsel oder dem Spielende. Auch diese Klasse steht in engem Austausch mit dem **ClientHandler**, um die vom Server empfangenen Nachrichten korrekt weiterzuverarbeiten.

(1) Protokoll

Message.java

In dieser Aufgabe sollen Sie die abstrakte Klasse **Message** im Package `protocol.messages` implementieren. Diese Klasse stellt die Basis für alle Nachrichtenobjekte dar, die über das Netzwerk zwischen Server und Client ausgetauscht werden. Die Klasse selbst enthält allgemeine Informationen, die für jeden Nachrichtentyp gültig sind.

Erweitern Sie zunächst die Klassendeklaration der Klasse **Message** so, dass sie explizit

als abstrakte Klasse definiert wird. Ergänzen Sie die Klasse außerdem um ein privates Attribut namens `type` vom Typ `MessageType`, das später eindeutig angibt, um welche konkrete Art von Nachricht es sich handelt. Initialisieren Sie dieses Attribut innerhalb des gegebenen Konstruktors mit dem übergebenen Parameter.

Ergänzen Sie anschließend eine öffentliche Getter-Methode (`getType()`), die es erlaubt, den Nachrichtentyp von außen abzufragen.

Beachten Sie, dass diese abstrakte Klasse als Grundlage für sämtliche weiteren konkreten Nachrichtentypen dient, weshalb hier keine weiteren Attribute oder spezifische Methoden für die Unterklassen enthalten sein sollen. Die tatsächlichen konkreten Nachrichtenklassen werden von dieser abstrakten Klasse abgeleitet und können bei Bedarf zusätzliche Attribute und Methoden definieren.

Stellen Sie abschließend sicher, dass Ihre Klasse das Interface `Serializable` implementiert.

RegisterMessage.java

In dieser Aufgabe implementieren Sie die Klasse `RegisterMessage` im Package `protocol.messages`. Diese Klasse dient dazu, einem Spieler die erfolgreiche **Registrierung** auf dem Server zu bestätigen. Sie enthält als wesentliche Informationen den **Benutzernamen** (username) und die eindeutige **Benutzer-ID** (userId) des Spielers.

Die Klasse `RegisterMessage` muss von der abstrakten Oberklasse `Message` erbe, um über Netzwerkstreams verschickt werden zu können.

Definieren Sie dazu zwei private, finale Attribute:

- Ein Attribut `username` vom Typ `String` für den Namen des registrierten Benutzers.
- Ein Attribut `userId` vom Typ `UUID` zur eindeutigen Identifikation des Benutzers.

Implementieren Sie einen **öffentlichen Konstruktor**, welcher die beiden Parameter `username` und `userId` erhält und diese Attribute entsprechend initialisiert. Zusätzlich müssen Sie im Konstruktor den Oberklassenkonstruktor mit dem Nachrichtentyp `REGISTER` explizit aufrufen.

Implementieren Sie außerdem je eine **Getter-Methode** für die Attribute, um diese nach außen zugänglich zu machen (`getUsername()` und `getUserId()`).

QueueUpdateMessage.java

In dieser Aufgabe implementieren Sie die Klasse `QueueUpdateMessage` im Package `protocol.messages.lobby`. Diese Klasse informiert den Spieler über den aktuellen Zustand der

Warteschlange, indem sie ihm mitteilt, wie groß die Warteschlange aktuell ist und ob er selbst Teil dieser Warteschlange ist. Diese Nachricht wird vom Server an den Client gesendet und erbt von der abstrakten Klasse `Message`.

Die Klasse `QueueUpdateMessage` muss von der Klasse `Message` erben. Erstellen Sie hierzu zunächst zwei private Attribute:

- `queueSize` vom Typ `int`: Gibt die aktuelle Größe der Warteschlange an.
- `playerInQueue` vom Typ `boolean`: Gibt an, ob der Spieler aktuell in der Warteschlange ist oder nicht.

Implementieren Sie anschließend einen Konstruktor, der genau diese beiden Attribute übergeben bekommt (`int queueSize`, `boolean playerInQueue`) und diese Werte entsprechend zuweist. Rufen Sie im Konstruktor zusätzlich den Konstruktor der Oberklasse `Message` mit dem Nachrichtentyp `QUEUE_UPDATE` auf.

Erstellen Sie zusätzlich **Getter-Methoden** für beide Attribute (`getQueueSize()` und `isPlayerInQueue()`), um externen Zugriff auf den jeweiligen Wert zu ermöglichen. Implementieren Sie schließlich die Methode `toString()`, welche eine klar lesbare String-Darstellung der `QueueUpdateMessage` mit den enthaltenen Informationen (`queueSize` und `playerInQueue`) zurückliefert.

LeaveQueueMessage.java

In dieser Aufgabe implementieren Sie die Klasse `LeaveQueueMessage` im Package `protocol.messages.lobby`. Diese Nachricht wird vom Client an den Server gesendet, wenn der Spieler die Warteschlange verlassen möchte.

Die Klasse `LeaveQueueMessage` muss von der abstrakten Klasse `Message` erben.

Implementieren Sie hierzu einen parameterlosen Konstruktor, welcher den Konstruktor der Oberklasse `Message` aufruft und diesem den Nachrichtentyp `LEAVE_QUEUE` übergibt.

JoinQueueMessage.java

In dieser Aufgabe implementieren Sie die Klasse `JoinQueueMessage` im Package `protocol.messages.lobby`. Diese Nachricht wird vom Client an den Server gesendet, sobald ein Spieler der Warteschlange beitreten möchte.

Die Klasse `JoinQueueMessage` muss von der abstrakten Klasse `Message` erben. Implementieren Sie hierzu einen parameterlosen Konstruktor, der ausschließlich den Konstruktor der Oberklasse `Message` aufruft und diesem den Nachrichtentyp `JOIN_QUEUE` übergibt.

JoinGameWithCodeMessage.java

In dieser Aufgabe implementieren Sie die Klasse `JoinGameWithCodeMessage` im Package `protocol.messages.lobby`. Diese Nachricht wird vom Client an den Server gesendet, wenn ein Spieler einem bestehenden Spiel mit Hilfe eines **sechsstelligen Session-Codes** beitreten möchte. Die Klasse `JoinGameWithCodeMessage` muss von der abstrakten Klasse `Message` erben.

Erstellen Sie zunächst ein `private` finales Attribut mit dem Namen `sessionCode` vom Typ `int`, das den sechsstelligen Session-Code enthält. Implementieren Sie anschließend einen öffentlichen Konstruktor, der genau einen Parameter (`int sessionCode`) erhält und das Attribut entsprechend initialisiert. Im Konstruktor muss zusätzlich der Konstruktor der Oberklasse `Message` mit dem Nachrichtentyp `JOIN_GAME_WITH_CODE` aufgerufen werden.

Erstellen Sie abschließend eine öffentliche Getter-Methode namens `getSessionCode()`, um den Zugriff auf das Attribut von außen zu ermöglichen.

CreateGameMessage.java

In dieser Aufgabe implementieren Sie die Klasse `CreateGameMessage` im Package `protocol.messages.lobby`. Diese Nachricht wird vom Client an den Server gesendet, wenn ein Spieler ein neues Spiel erstellen möchte. Sie enthält als wichtige Information die gewünschten Spielooptionen (gespeichert in einem Objekt vom Typ `GameOptions`).

Die Klasse `CreateGameMessage` muss von der abstrakten Klasse `Message` erben.

Fügen Sie zunächst ein `private` Attribut namens `gameOptions` vom Typ `GameOptions` hinzu, um die Spielooptionen zu speichern. Erstellen Sie anschließend einen öffentlichen Konstruktor, der ein Objekt vom Typ `GameOptions` als Parameter erhält und dieses Attribut entsprechend initialisiert. Rufen Sie dabei im Konstruktor ebenfalls den Konstruktor der Oberklasse `Message` mit dem Nachrichtentyp `CREATE_GAME` auf.

Implementieren Sie zusätzlich eine öffentliche Getter-Methode namens `getGameOptions()`, welche den Zugriff auf das Attribut `gameOptions` ermöglicht.

LeaveGameMessage.java

In dieser Aufgabe implementieren Sie die Klasse `LeaveGameMessage` im Package `protocol.messages.game`. Diese Nachricht wird vom Client an den Server gesendet, wenn ein Spieler ein laufendes Spiel verlassen möchte. Diese Nachricht wird genutzt, um den Server darüber zu informieren, dass der Client seine Spielsitzung beendet. Die Klasse `LeaveGameMessage` muss von der abstrakten Klasse `Message` erben.

Implementieren Sie hierzu einen **öffentlichen, parameterlosen Konstruktor**. Dieser Konstruktor ruft ausschließlich den Konstruktor der Oberklasse (`Message`) explizit mit dem Nachrichtentyp `LEAVE_GAME` auf.

PlayerMoveMessage.java

In dieser Aufgabe implementieren Sie die Klasse `PlayerMoveMessage` im Package `protocol.messages.game.ingame`. Diese Nachricht wird vom **Client an den Server** gesendet, sobald ein Spieler während des laufenden Spiels einen **Spielzug** (Move) ausführt. Die Klasse `PlayerMoveMessage` erbt von der abstrakten Klasse `Message`.

Definieren Sie zunächst ein `privates, finales` Attribut namens `move` vom Typ `Move`, das den vom Spieler ausgeführten Spielzug speichert.

Implementieren Sie anschließend einen **öffentlichen Konstruktor**, welcher ein Objekt vom Typ `Move` als Parameter erhält und das Attribut entsprechend initialisiert. Der Konstruktor muss zusätzlich den Oberklassenkonstruktor explizit mit dem Nachrichtentyp `PLAYER_MOVE` aufrufen.

Implementieren Sie außerdem eine öffentliche Getter-Methode namens `getMove()`, um externen Zugriff auf den ausgeführten Spielzug zu ermöglichen.

PlayerHoverMessage.java

In dieser Aufgabe implementieren Sie die Klasse `PlayerHoverMessage` im Package `protocol.messages.game.ingame`. Diese Nachricht wird versendet, wenn ein Spieler im laufenden Spiel mit der Maus über ein bestimmtes Spielfeld (eine Zelle) fährt („hovering“).

Dabei tritt die Nachricht in zwei Kontexten auf:

- Als Nachricht vom **Client an den Server**, wenn der Spieler über eine Zelle fährt.
- Als Nachricht vom **Server an den Client**, um zu signalisieren, dass der Gegner über eine Zelle fährt.

Die Klasse `PlayerHoverMessage` erbt von der abstrakten Klasse `Message`.

Definieren Sie zunächst folgende **private Attribute**:

- `userId` vom Typ `UUID`: die ID des Spielers, der das Hover-Ereignis ausgelöst hat.
- `x` vom Typ `int`: die X-Koordinate der Zelle.
- `y` vom Typ `int`: die Y-Koordinate der Zelle.
- `affectedFields` vom Typ `ArrayList<Cell>`: Liste der Zellen, die durch das Hover-Ereignis betroffen sind.

Implementieren Sie anschließend zwei **Konstruktoren**:

1. Einen Konstruktor mit den Parametern `UUID userId`, `int x`, `int y`, und `ArrayList<Cell> affectedFields`. Dieser Konstruktor initialisiert alle Attribute mit den übergebenen Werten und ruft explizit den Konstruktor der Oberklasse `Message` mit dem Nachrichtentyp `PLAYER_HOVER` auf.
2. Einen Kopierkonstruktor, der als Parameter ein bereits existierendes Objekt vom Typ `PlayerHoverMessage` erhält. Dieser initialisiert die Attribute entsprechend der Werte des übergebenen Objekts und ruft ebenfalls den Oberklassenkonstruktor mit dem Typ `PLAYER_HOVER` auf.

Implementieren Sie außerdem die öffentlichen Getter-Methoden `getUserId()`, `getX()`, `getY()` und `getAffectedFields()`, um externen Zugriff auf die Attribute zu ermöglichen.

GameInGameStartMessage.java

In dieser Aufgabe implementieren Sie die Klasse `GameInGameStartMessage` im Package `protocol.messages.game.ingame`. Diese Nachricht wird vom **Server an den Client** gesendet, sobald die aktive Spielphase beginnt. Der Server versendet diese Nachricht entweder, nachdem beide Spieler ihre Bereitschaft bestätigt haben (`PlayerReadyMessage`), oder sobald die Zeit für die Bauphase abgelaufen ist (`GameBuildingStartMessage`). Die Klasse `GameInGameStartMessage` erbt von der abstrakten Klasse `Message`.

Definieren Sie zunächst zwei **private Attribute**:

- `gameState` vom Typ `GameState`: Beschreibt den aktuellen Zustand des Spiels zu Beginn der aktiven Spielphase.
- `yourShips` vom Typ `ArrayList<Ship>`: Enthält die Liste der Schiffe des aktuellen Spielers.

Implementieren Sie anschließend einen **Konstruktor** mit den Parametern (`GameState gameState`, `ArrayList<Ship> yourShips`). Im Konstruktor weisen Sie den beiden Attributen die Parameter entsprechend zu. Dabei erstellen Sie für die Schiffsliste eine **neue Kopie**, um eine direkte externe Änderung der Schiffsliste zu vermeiden. Im Konstruktor rufen Sie zusätzlich explizit den Oberklassenkonstruktor mit dem Nachrichtentyp `GAME_IN_GAME_START` auf.

Implementieren Sie zusätzlich **öffentliche Getter-Methoden** für beide Attribute (`getGameState()` und `getYourShips()`), um externen Zugriff darauf zu ermöglichen.

PlayerUpdateShipPlacement.java

In dieser Aufgabe implementieren Sie die Klasse `PlayerUpdateShipPlacement` im Package `protocol.messages.game.building`. Diese Nachricht wird vom Client an den Server gesendet, sobald ein Spieler während der Bauphase („Building-Phase“) des Spiels eine **neue Positionierung seiner Schiffe** mitteilt. Die Klasse `PlayerUpdateShipPlacement` muss von der abstrakten Oberklasse `Message` erben.

Erstellen Sie zunächst ein `private`, `finale`s Attribut namens `ships` vom Typ `ArrayList<Ship>`, das die aktuell platzierten Schiffe des Spielers beinhaltet.

Implementieren Sie anschließend einen öffentlichen Konstruktor, der genau diese Schiffsliste (`ArrayList<Ship>`) als Parameter erhält und damit das Attribut initialisiert. Im Konstruktor rufen Sie zusätzlich explizit den Konstruktor der Oberklasse `Message` mit dem Nachrichtentyp `PLAYER_UPDATE_SHIP_PLACEMENT` auf.

Implementieren Sie zusätzlich eine öffentliche Getter-Methode `getShips()`, um externen Zugriff auf die aktuelle Schiffsliste zu ermöglichen.

PlayerReadyMessage.java

In dieser Aufgabe implementieren Sie die Klasse `PlayerReadyMessage` im Package `protocol.messages.game.building`. Diese Nachricht wird vom Client an den Server gesendet, wenn ein Spieler seinen **Bereitschaftsstatus** (`ready`) mitteilt, um dem Server zu signalisieren, dass er bereit ist, das Spiel zu starten. Die Klasse `PlayerReadyMessage` muss von der abstrakten Oberklasse `Message` erben.

Definieren Sie hierzu ein `private`, `finale`s Attribut namens `ready` vom Typ `boolean`, welches den aktuellen Bereitschaftsstatus des Spielers speichert.

Implementieren Sie anschließend einen **öffentlichen Konstruktor**, welcher den Bereitschaftsstatus (`boolean ready`) als Parameter erhält und dieses Attribut entsprechend initialisiert. Im Konstruktor muss zusätzlich der Oberklassenkonstruktor explizit mit dem Nachrichtentyp `PLAYER_READY` aufgerufen werden.

Implementieren Sie zusätzlich eine öffentliche Getter-Methode namens `isReady()`, welche externen Zugriff auf das Attribut `ready` erlaubt.

GameState-basierte Message-Klassen

In dieser Aufgabe implementieren Sie mehrere `Message`-Klassen, die alle eine ähnliche Struktur aufweisen und sich lediglich im **Namen der Klasse** sowie im übergebenen **MessageType** unterscheiden. Jede dieser Klassen wird verwendet, um den Client über den aktuellen Spielzustand (`GameState`) zu informieren. Alle Klassen erben von der ab-

strakten Klasse `Message`.

Implementieren Sie für jede Klasse ein `private`, `finale` Attribut namens `gameState` vom Typ `GameState`. Erstellen Sie anschließend einen öffentlichen Konstruktor, der genau einen Parameter vom Typ `GameState` erhält, dieses Attribut initialisiert und dabei explizit den Konstruktor der Oberklasse mit dem jeweils passenden `MessageType` aufruft.

Ergänzen Sie jede Klasse um eine `Getter`-Methode (`getGameState()`), um Zugriff auf den Spielzustand zu ermöglichen. Überschreiben Sie abschließend die Methode `toString()`, um eine sinnvolle String-Repräsentation der Nachricht sicherzustellen.

Die einzelnen Klassen unterscheiden sich nur durch ihren Namen und den jeweils zu verwendenden `MessageType`, der wie folgt festgelegt ist:

Klassenname	Zu verwendender MessageType
<code>protocol.messages.game.GameOverMessage.java</code>	<code>GAME_OVER</code>
<code>protocol.messages.game.JoinGameMessage.java</code>	<code>JOIN_GAME</code>
<code>protocol.messages.game.building.PlayerReadyStateChangeMessage.java</code>	<code>BUILD_READY_STATE_CHANGE</code>
<code>protocol.messages.game.ingame.MoveMadeMessage.java</code>	<code>MOVE_MADE</code>
<code>protocol.messages.game.ingame.PlayerTurnChangeMessage.java</code>	<code>TURN_CHANGE</code>
<code>protocol.messages.game.building.GameBuildingStartMessage.java</code>	<code>BUILDING_PHASE_STARTS</code>

(2) Server

`Server.java`

Im Folgenden soll die parallele Bearbeitung der eingehenden Nachrichten vom Client auf dem Server in der Datei `server.Server.java` mithilfe von Threads implementiert werden. Dabei soll jeder Client sowie jedes Spiel in einem eigenen Thread verarbeitet werden, sodass mehrere Spieler gleichzeitig bedient werden können.

Implementieren Sie wichtige Datenstrukturen innerhalb der Klasse `Server`, die dazu dienen, Spieler, aktive Spielsitzungen und die Warteschlange zu verwalten.

Implementieren Sie dazu folgende **private**, **finale** Attribute:

- **players**: Eine Liste (`List<ServerPlayer>`), in der alle aktuell verbundenen Spieler gespeichert werden.
- **clientThreads**: Eine Liste (`List<Thread>`), in der alle Threads gespeichert werden, die für die Kommunikation mit den einzelnen Clients zuständig sind.
- **games**: Eine Map (`Map<UUID, GameContainer>`), welche die aktiven Spiele verwaltet und jedem Spiel eine eindeutige ID (`UUID`) zuordnet.
- **queue**: Eine Liste (`ArrayList<ServerPlayer>`), in der alle Spieler gespeichert werden, die sich aktuell in der Warteschlange befinden.

Initialisieren Sie alle Attribute direkt bei der Deklaration mit einer passenden leeren Datenstruktur Ihrer Wahl.

startServer()

Implementieren Sie die Methode `startServer()`, welche den Server initialisiert und startet, sodass neue Client-Verbindungen akzeptiert und verwaltet werden können.

Überprüfen Sie zu Beginn, ob der Server bereits läuft, indem Sie das Attribut `running` prüfen. Falls der Server bereits läuft, verlassen Sie die Methode direkt.

Erstellen und starten Sie anschließend einen neuen Thread, in welchem Sie zunächst einen neuen `ServerSocket` auf dem definierten Port (`PORT`) initialisieren. Setzen Sie nach erfolgreicher Initialisierung das Attribut `running` auf `true` und aktualisieren Sie den Online-Status in der GUI mit `gui.updateServerOnlineStatus(true)`.

Implementieren Sie innerhalb des Threads anschließend eine Schleife, welche dauerhaft ausgeführt wird, solange das Attribut `running` auf `true` steht. Nutzen Sie die Methode `serverSocket.accept()`, um eingehende Client-Verbindungen anzunehmen.

Für jede angenommene Client-Verbindung erstellen Sie eine neue Instanz von `ServerPlayer` (mit den Parametern `clientSocket` und einer Referenz auf die Serverinstanz) und fügen diese der Liste `players` hinzu. Erstellen Sie außerdem einen eigenen Thread für diesen Spieler und starten Sie diesen Thread. Fügen Sie den gestarteten Thread zur Liste `clientThreads` hinzu.

Zum Schluss rufen Sie nach jeder angenommenen Verbindung die Methode `updatePlayerList()` auf, um die GUI oder andere Komponenten über die neuen Spielerinformationen zu informieren.

Achten Sie darauf, mögliche Ausnahmen (`IOException`) angemessen abzufangen.

registerGame(BattleShipGame game)

In dieser Aufgabe implementieren Sie die Methode `registerGame(BattleShipGame game)`, die dafür verantwortlich ist, ein neues Spiel auf dem Server zu registrieren und zu starten.

Erstellen Sie zunächst einen neuen `Thread`, der das übergebene `BattleShipGame` ausführt. Dieser `Thread` sorgt dafür, dass das Spiel unabhängig vom Haupt-`Thread` des Servers läuft. Anschließend erzeugen Sie ein neues Objekt vom Typ `GameContainer`, das sowohl das Spiel als auch den dazugehörigen `Thread` speichert.

Speichern Sie dieses `GameContainer`-Objekt dann in der Map `games`, wobei die eindeutige ID des Spiels als Schlüssel verwendet wird. Diese ID wird über die `GameState`-Instanz des Spiels abgerufen und sorgt dafür, dass jedes Spiel eindeutig identifiziert werden kann.

Starten Sie anschließend den Spiel-`Thread`, damit das Spiel beginnt. Zum Schluss rufen Sie die Methode `updateGameList()` auf, um die Liste der laufenden Spiele zu aktualisieren.

unregisterGame(UUID id)

Implementieren Sie die Methode `unregisterGame(UUID id)`, mit der ein laufendes Spiel anhand seiner eindeutigen ID vom Server entfernt werden kann.

Greifen Sie zunächst über die übergebene `UUID` auf das entsprechende Spiel in der Map `games` zu und speichern Sie das zugehörige `GameContainer`-Objekt in einer lokalen Variable. Überprüfen Sie dann, ob dieses Objekt existiert (also `nicht null` ist).

Falls das Spiel existiert, unterbrechen Sie den zugehörigen Spiel-`Thread` durch Aufruf von `container.getThread().interrupt()`, sodass das Spiel korrekt gestoppt wird. Anschließend entfernen Sie das Spiel über `games.remove(id)` aus der Map.

Zum Schluss rufen Sie die Methode `updateGameList()` auf, um die Liste der laufenden Spiele zu aktualisieren.

getGame(ServerPlayer player)

In dieser Aufgabe implementieren Sie die Methode `getGame(ServerPlayer player)`, mit der ein Spieler seinem aktuellen Spiel zugeordnet werden kann.

Durchsuchen Sie dafür alle aktuell laufenden Spiele, die in der Map `games` gespeichert sind. Iterieren Sie über die Werte dieser Map (die `GameContainer`-Objekte). Über jedes `GameContainer`-Objekt greifen Sie mit `getGame()` auf das enthaltene `BattleShipGame` zu.

Überprüfen Sie dann, ob das Spiel überhaupt existiert (also **nicht null** ist). Wenn das Spiel existiert, prüfen Sie, ob entweder **Spieler A** oder **Spieler B** dieses Spiels die gleiche ID wie der übergebene Spieler besitzt. Wenn dies der Fall ist, geben Sie dieses Spiel zurück.

Falls kein passendes Spiel gefunden wird, geben Sie am Ende der Methode **null** zurück.

Vergessen Sie nicht, die Methode als **synchronized** zu markieren, damit ein gleichzeitiger Zugriff durch mehrere Threads (Spieler) verhindert wird.

getGameFromJoinCode(int joinCode)

Implementieren Sie die Methode `getGameFromJoinCode(int joinCode)`, mit der ein Spiel anhand seines Join-Codes gefunden werden kann.

Durchlaufen Sie dafür alle aktuell laufenden Spiele, die in der Map `games` gespeichert sind. Greifen Sie über jedes `GameContainer`-Objekt mit `getGame()` auf das zugehörige Spiel (`BattleShipGame`) zu. Über die Methode `getGameState().getSessionCode()` erhalten Sie den Join-Code des Spiels.

Vergleichen Sie diesen mit dem als Parameter übergebenen Join-Code. Falls ein Spiel mit passendem Join-Code gefunden wurde, geben Sie dieses Spiel direkt zurück. Sollte kein Spiel mit diesem Join-Code existieren, geben Sie am Ende der Methode **null** zurück.

addToQueue(ServerPlayer player)

In dieser Aufgabe implementieren Sie die Methode `addToQueue(ServerPlayer player)`, mit der ein Spieler zur Matchmaking-Warteschlange (`queue`) hinzugefügt werden kann.

Fügen Sie den übergebenen Spieler zunächst zur Liste `queue` hinzu. Anschließend senden Sie dem Spieler eine `QueueUpdateMessage` (`player.sendMessage(...)`), um ihm die aktuelle Größe der Warteschlange mitzuteilen und zu signalisieren, dass er nun in der Warteschlange ist. Dabei übergeben Sie als Parameter der Nachricht die aktuelle Größe der Warteschlange und den Wert **true**.

removeFromQueue(UUID id)

Implementieren Sie die Methode `removeFromQueue(UUID id)`, mit der ein Spieler anhand seiner ID aus der Warteschlange entfernt wird.

Suchen Sie in der Warteschlange (`queue`) nach einem Spieler, dessen ID mit der übergebenen UUID übereinstimmt, und entfernen Sie diesen Spieler aus der Warteschlange. Nachdem der Spieler entfernt wurde, soll diesem Spieler eine `QueueUpdateMessage` gesendet

werden, um ihn darüber zu informieren, dass er sich **nicht mehr** in der Warteschlange befindet. Die Nachricht soll zusätzlich die **aktuelle Größe der Warteschlange** enthalten.

(3) Server

Implementieren Sie in der Klasse `ServerPlayer` (in `server.ServerPlayer.java`) die Methode `run()`, welche die grundlegende Kommunikation zwischen Server und Client regelt. Zu Beginn der Methode initialisieren Sie zunächst die erforderlichen Streams (`ObjectOutputStream` und `ObjectInputStream`) für die Kommunikation. Anschließend senden Sie dem Client unmittelbar nach der Initialisierung zwei Nachrichten zur Bestätigung der Verbindung: eine `RegisterMessage` sowie eine `QueueUpdateMessage` mit der aktuellen Größe der Warteschlange.

Danach implementieren Sie eine **Endlosschleife**, die so lange ausgeführt wird, wie die Verbindung zum Client aktiv ist (`socket.isConnected()`). Innerhalb dieser Schleife empfangen Sie Nachrichten vom Client, ermitteln deren Typ mittels `getType()`, und reagieren entsprechend. Verwenden Sie dafür eine **switch-Anweisung**, die jeden eingehenden `MessageType` separat behandelt:

Falls die Methode `run()` eine `IOException` oder `ClassNotFoundException` wirft, rufen Sie die Methoden `removePlayer(this)` und `removeFromQueue(this.getId())` des `Server`-Objekts `server` auf, um den Client zu trennen und sicherzustellen, dass er aus der Warteschlange entfernt wird.

JOIN_QUEUE

In dieser Aufgabe soll die Verarbeitung einer `JOIN_QUEUE`-Nachricht umgesetzt werden. Zuerst prüfen Sie, ob der Spieler bereits in einem Spiel ist, indem Sie die Methode `server.getGame(...)` verwenden. Falls ja, senden Sie eine `ErrorMessage` mit dem `ErrorType.ALREADY_IN_GAME` an den Spieler und beenden den Vorgang. Wenn der Spieler bereits in der Warteschlange enthalten ist, prüfen Sie dies mit `server.getQueue().contains(...)`. Falls dies zutrifft, senden Sie mit `broadcastQueueStateUpdate()` ein Update an alle Clients und beenden den Vorgang. Wenn der Spieler weder in einem Spiel noch in der Warteschlange ist, fügen Sie ihn mit `server.addToQueue(this)` der Warteschlange hinzu. Danach senden Sie ein `broadcastQueueStateUpdate()`, um alle Spieler über die neue Warteschlange zu informieren. Zum Schluss prüfen Sie, ob ein neues Spiel gestartet werden kann, indem Sie die Methode `checkQueueForPossibleGame()` aufrufen.

LEAVE_QUEUE

In dieser Aufgabe soll die Verarbeitung einer `LEAVE_QUEUE`-Nachricht umgesetzt werden. Zuerst prüfen Sie, ob der Spieler bereits in einem Spiel ist, indem Sie die Me-

thode `server.getGame(...)` aufrufen. Ist das der Fall, entfernen Sie den Spieler mit `game.removePlayer(this)` aus dem Spiel, senden eine `ErrorMessage` (`sendMessage(...)`) mit dem Fehler `ALREADY_IN_GAME` an den Spieler und beenden den Vorgang. Falls der Spieler **nicht** in der Warteschlange ist, prüfen Sie dies mit `server.getQueue().contains(...)`. Wenn der Spieler nicht enthalten ist, rufen Sie die Methode `broadcastQueueStateUpdate()` auf und beenden den Vorgang. Falls der Spieler in der Warteschlange ist, entfernen Sie ihn mit `server.removeFromQueue(...)` und senden danach ein `broadcastQueueStateUpdate()`, um alle Clients über den neuen Zustand der Warteschlange zu informieren.

CREATE_GAME

In dieser Aufgabe soll die Verarbeitung einer `CREATE_GAME`-Nachricht implementiert werden. Zuerst casten Sie die empfangene Nachricht zu `CreateGameMessage`, um auf die Spieloptionen zuzugreifen. Anschließend prüfen Sie mit `createGameMessage.getGameOptions().getBoardSize()`, ob die Spielfeldgröße gültig ist. Die Boardgröße muss dabei zwischen 5 und 20 liegen. Ist die Größe ungültig, senden Sie eine `ErrorMessage` mit dem Fehler `INVALID_GAME_SIZE` an den Spieler und brechen die Verarbeitung ab. Wenn die Größe korrekt ist, starten Sie ein neues Spiel mit `createGame(...)` und übergeben die Game-Options, entfernen den Spieler aus der Warteschlange (`server.getQueue()`) und setzen das Flag `isInGame` auf `true`, um zu markieren, dass der Spieler nun in einem Spiel ist.

JOIN_GAME_WITH_CODE

In dieser Aufgabe soll die Verarbeitung einer `JOIN_GAME_WITH_CODE`-Nachricht implementiert werden. Zuerst casten Sie die empfangene Nachricht zu `JoinGameWithCodeMessage`, um den Session-Code zu erhalten. Anschließend prüfen Sie, ob der Spieler bereits in einem Spiel ist, und senden ihm in diesem Fall eine `ErrorMessage` mit dem Fehler `ALREADY_IN_GAME`. Wenn nicht, rufen Sie die Methode `server.getGameFromJoinCode(...)` auf, um das Spiel mit dem übergebenen Code zu finden. Existiert kein Spiel mit diesem Code, senden Sie eine `ErrorMessage` mit `INVALID_SESSION_CODE` zurück. Falls das Spiel bereits gestartet wurde (Status ungleich `LOBBY_WAITING`), wird eine weitere `ErrorMessage` mit `GAME_ALREADY_STARTED` gesendet. Wenn das Spiel korrekt gefunden und noch nicht gestartet ist, entfernen Sie den Spieler aus der Warteschlange im Server, fügen ihn dem Spiel mittels `targetGame.addPlayer(this)` hinzu und setzen das Flag `isInGame` auf `true`.

LEAVE_GAME

In dieser Aufgabe soll die Verarbeitung einer `LEAVE_GAME`-Nachricht umgesetzt werden. Zuerst casten Sie die empfangene Nachricht zu `LeaveGameMessage`. Danach überprüfen Sie, ob der Spieler aktuell in einem Spiel ist. Falls nicht, senden Sie eine `ErrorMessage` mit dem Fehler `NO_GAME_IN_PROGRESS` zurück. Falls der Spieler sich in einem Spiel befindet, rufen Sie `game.removePlayer(...)` auf, um ihn aus dem Spiel zu entfernen, und setzen anschließend das Attribut `isInGame` auf `false`.

PLAYER_UPDATE_SHIP_PLACEMENT

In dieser Aufgabe soll die Verarbeitung einer `PLAYER_UPDATE_SHIP_PLACEMENT`-Nachricht implementiert werden. Zuerst casten Sie die empfangene Nachricht zu `PlayerUpdateShipPlacement`, um die gesetzten Schiffe des Spielers zu erhalten.

Anschließend überprüfen Sie, ob der Spieler aktuell Teil eines Spiels ist. Ist dies **nicht** der Fall, senden Sie dem Spieler eine `ErrorMessage` mit dem Fehler `NO_GAME_IN_PROGRESS` zurück und beenden die Verarbeitung der Nachricht.

Falls der Spieler in einem Spiel ist, rufen Sie die Methode `game.onPlayerPlaceShips(...)` auf, um die Schiffsdaten im Spiel zu registrieren.

PLAYER_HOVER

Zuerst casten Sie die empfangene Nachricht zu `PlayerHoverMessage`, um die Hover-Informationen des Spielers zu erhalten. Anschließend prüfen Sie, ob der Spieler aktuell in einem Spiel ist. Falls **kein Spiel** aktiv ist, senden Sie eine `ErrorMessage` mit dem Fehler `NO_GAME_IN_PROGRESS` zurück und beenden die Verarbeitung.

Falls der Spieler sich in einem laufenden Spiel befindet, wird die Hover-Nachricht an den jeweils anderen Spieler weitergeleitet.

PLAYER_MOVE

Zuerst casten Sie die empfangene Nachricht zu `PlayerMoveMessage`, um den Spielzug des Spielers zu erhalten. Falls kein Spiel aktiv ist, senden Sie eine `ErrorMessage` mit dem Fehler `NO_GAME_IN_PROGRESS` zurück. Falls der Spieler in einem Spiel ist, übergeben Sie den Spielzug mit der Methode `game.onPlayerAttemptMove(...)` an das Spiel.

sendMessage(Message message)

In dieser Aufgabe implementieren Sie die Methode `sendMessage(Message message)`, mit der Nachrichten an den jeweiligen Spieler gesendet werden. Senden Sie die Nachricht über den `ObjectOutputStream` und rufen Sie anschließend `flush()` auf.

(4) Client

ClientHandler.java

Implementieren Sie den Konstruktor der Klasse `ClientHandler` (in `client.Client.java`), der eine Verbindung zum Server herstellt.

Der Konstruktor erhält die Parameter `serverAddress` (Server-Adresse) und `port` (Server-Port). Erzeugen Sie damit zunächst eine `Socket`-Verbindung zum Server.

Nach dem erfolgreichen Aufbau der Verbindung initialisieren Sie einen `ObjectOutputStream` und einen `ObjectInputStream`, welche für die Kommunikation mit dem Server benötigt werden.

Danach erstellen Sie die Instanzen für den `LobbyHandler` und den `StageManager`, die jeweils eine Referenz auf den aktuellen `ClientHandler` erhalten sollen.

Starten Sie abschließend die Methode `listenForMessages()`, um auf eingehende Nachrichten des Servers reagieren zu können.

Falls beim Aufbau der Verbindung oder bei der Stream-Initialisierung ein Fehler auftritt (z.B. eine `IOException`), informieren Sie den Benutzer über den Fehler und beenden Sie anschließend das Programm.

listenForMessages()

Implementieren Sie die Methode `listenForMessages()`, um kontinuierlich Nachrichten des Servers zu empfangen und zu verarbeiten.

Starten Sie zunächst einen neuen Thread, um die Kommunikation mit dem Server unabhängig vom restlichen Programm ablaufen zu lassen. Innerhalb des Threads implementieren Sie eine Schleife, die solange ausgeführt wird, wie die Socket-Verbindung besteht. In der Schleife lesen Sie Objekte über den `ObjectInputStream` ein und casten diese in den abstrakten Typ `Message`. Anhand des in jeder Nachricht enthaltenen `MessageType` ermitteln Sie nun, wie diese Nachricht verarbeitet werden soll.

Implementieren Sie nun im zweiten Teil der Aufgabe eine **Switch-Anweisung**, die je nach `MessageType` unterschiedliche Abläufe vorsieht. Hier sind die Anleitungen für alle `MessageTypes` aus Ihrem Code, im gleichen Stil wie Ihr Beispiel:

REGISTER

In dieser Aufgabe soll die Verarbeitung einer `REGISTER`-Nachricht implementiert werden. Zuerst casten Sie die empfangene Nachricht zu `RegisterMessage`, um den Benutzernamen und die Benutzer-ID zu erhalten. Speichern Sie diese Daten anschließend in den Attributen `username` und `userId` des Clients. Zum Schluss aktualisieren Sie die GUI mit dem Benutzernamen, indem Sie `stageManager.lobbyScene.setUsername(username)` aufrufen.

QUEUE_UPDATE

In dieser Aufgabe soll die Verarbeitung einer `QUEUE_UPDATE`-Nachricht implementiert werden. Zuerst casten Sie die empfangene Nachricht zu `QueueUpdateMessage`, um die ak-

tuellen Informationen zur Warteschlange zu erhalten. Diese Informationen geben Sie anschließend über die Methode `lobbyHandler.onQueueUpdate(...)` an den `LobbyHandler` weiter, damit die GUI entsprechend aktualisiert wird.

ERROR

In dieser Aufgabe soll die Verarbeitung einer **ERROR**-Nachricht implementiert werden. Zuerst casten Sie die empfangene Nachricht zu `ErrorMessage`, um den enthaltenen Fehler auszulesen. Falls der Fehler den Typ `SERVER_CLOSED` hat, informieren Sie den Benutzer mit einem Dialogfenster und beenden das Programm. Handelt es sich um einen anderen Fehler, prüfen Sie, ob der `gameHandler` existiert. Falls ja, rufen Sie dort `onGameError(...)` auf, um den Fehler im Spiel zu behandeln. Falls kein Spiel läuft, delegieren Sie den Fehler an den `LobbyHandler` über `onLobbyError(...)`.

JOIN_GAME

In dieser Aufgabe soll die Verarbeitung einer **JOIN_GAME**-Nachricht implementiert werden. Zuerst casten Sie die empfangene Nachricht zu `JoinGameMessage`, um die Informationen zur Spielsitzung zu erhalten. Falls bereits ein Spiel läuft, senden Sie zunächst eine Nachricht an den Server, um das aktuelle Spiel zu verlassen. Anschließend erstellen Sie mit `new GameHandler(...)` eine neue Spielinstanz und übergeben die empfangenen Spieldaten. Zum Schluss starten Sie die Wartelobby mit `stageManager.startWaitingLobbyScene()` und setzen `lobbyHandler.setInQueue(false)`, um den Spieler aus der Warteschlange zu entfernen.

BUILDING_PHASE_STARTS

In dieser Aufgabe soll die Verarbeitung einer **BUILDING_PHASE_STARTS**-Nachricht implementiert werden. Zuerst casten Sie die empfangene Nachricht zu `GameBuildingStartMessage`, um die Daten für die Bauphase zu erhalten. Prüfen Sie anschließend, ob ein Spiel läuft. Falls ja, starten Sie die Bauphase mit `gameHandler.onBuildPhaseStarts(...)` und übergeben die empfangenen Daten.

GAME_IN_GAME_START

In dieser Aufgabe soll die Verarbeitung einer **GAME_IN_GAME_START**-Nachricht implementiert werden. Zuerst casten Sie die empfangene Nachricht zu `GameInGameStartMessage`, um die Startdaten des Spiels zu erhalten. Prüfen Sie anschließend, ob ein Spiel läuft. Falls ja, starten Sie das eigentliche Spiel mit `gameHandler.onGameInGameStarts(...)`.

GAME_OVER

In dieser Aufgabe soll die Verarbeitung einer **GAME_OVER**-Nachricht implementiert werden. Zuerst casten Sie die empfangene Nachricht zu `GameOverMessage`, um die Daten zum Spielende zu erhalten. Prüfen Sie anschließend, ob ein Spiel läuft. Falls ja, beenden

Sie das Spiel mit `gameHandler.onGameOver(...)` und aktualisieren den Endzustand in der GUI.

MOVE_MADE

In dieser Aufgabe soll die Verarbeitung einer `MOVE_MADE`-Nachricht implementiert werden. Zuerst casten Sie die empfangene Nachricht zu `MoveMadeMessage`, um die Informationen zum gemachten Spielzug zu erhalten. Prüfen Sie anschließend, ob ein Spiel läuft. Falls ja, übergeben Sie den Spielzug zur Verarbeitung an `gameHandler.onMoveMade(...)`.

BUILD_READY_STATE_CHANGE

In dieser Aufgabe soll die Verarbeitung einer `BUILD_READY_STATE_CHANGE`-Nachricht implementiert werden. Zuerst casten Sie die empfangene Nachricht zu `PlayerReadyStateChangeMessage`, um den neuen Bereitschaftsstatus des Spielers zu erhalten. Prüfen Sie anschließend, ob ein Spiel läuft. Falls ja, aktualisieren Sie den Status mit `gameHandler.onBuildReadyStateChange(...)`.

TURN_CHANGE

In dieser Aufgabe soll die Verarbeitung einer `TURN_CHANGE`-Nachricht implementiert werden. Zuerst casten Sie die empfangene Nachricht zu `PlayerTurnChangeMessage`, um die Information über den Spielerwechsel zu erhalten. Prüfen Sie anschließend, ob ein Spiel läuft. Falls ja, rufen Sie `gameHandler.onTurnChange(...)` auf, um den neuen aktiven Spieler zu setzen.

PLAYER_HOVER

In dieser Aufgabe soll die Verarbeitung einer `PLAYER_HOVER`-Nachricht implementiert werden. Zuerst casten Sie die empfangene Nachricht zu `PlayerHoverMessage`, um die Hover-Daten zu erhalten. Prüfen Sie anschließend, ob ein Spiel läuft. Falls ja, übergeben Sie die Informationen an `gameHandler.onPlayerHoverEvent(...)`, damit die GUI die Aktion anzeigt.

Starten des Servers und der Clients

Wenn Sie alle Aufgaben aus (1), (2), (3) und (4) erfolgreich implementiert haben, können Sie den Server in der Klasse `server.Server.java` starten. Zusätzlich können zwei Clients über die Klasse `client.Client.java` gestartet werden, um die Kommunikation und den Spielablauf zu testen. Der standard Port für die Verbindung ist 12345. Folge den Anweisungen in der Server Konsolen-Ausgabe, um das Spiel innerhalb eines Netzwerks zu spielen.